

Università degli Studi di Napoli Federico II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione

Corso di Laurea in Ingegneria Informatica

Il Dottore sotto esame:
Machine Learning applicato alle offerte
di *Affari Tuoi*

Elaborato Finale

Corso di **Elementi di Intelligenza Artificiale** 2025/2026

Prof. Giancarlo Sperli

Studente: Crisci Francesco Pio

Matricola: N46007601

Studente: Raia Umberto

Matricola: N46007558

github.com/crrisci/dottore-affarituoi

Indice

1	Introduzione	1
2	Il Machine Learning	1
2.1	Tipologie di apprendimento	1
2.2	Classificazione e regressione	2
2.3	Le fasi del processo ML	2
3	Il Dataset	3
3.1	Fonte e raccolta dei dati	3
3.2	Struttura e statistiche descrittive	3
3.3	Analisi esplorativa	4
4	Feature Engineering e Pre-processing	5
4.1	Estrazione delle feature	5
4.2	Costruzione della variabile target	6
4.3	Normalizzazione e suddivisione del dataset	6
5	Modelli di Classificazione	7
5.1	Decision Tree	7
5.2	Random Forest	7
5.3	Naive Bayes	8
6	Implementazione	8
6.1	Implementazione Python	8
6.2	Implementazione KNIME	9
7	Risultati e Valutazione	11
7.1	Metriche di valutazione	11
7.2	Confronto tra modelli	12
7.3	Analisi per classe	13
7.4	Feature importance	14
8	Interpretazione: quanto trattiene il Dottore?	14
8.1	Quanto è distante l'offerta dalla matematica?	14
8.2	Implicazioni pratiche	15
9	Conclusioni	16

1. Introduzione

Affari Tuoi è il programma televisivo più visto d'Italia nel 2024: ogni sera oltre sei milioni di spettatori seguono un concorrente alle prese con ventuno pacchi, ciascuno contenente un premio in denaro che va da 0 € a 300.000 €. Al centro dello show, letteralmente e metaforicamente, c'è il **Dottore**: un personaggio misterioso che, a intervalli regolari, telefona per proporre al concorrente una somma di denaro in cambio della rinuncia al proprio pacco.

Il programma ha scatenato negli ultimi mesi un acceso dibattito pubblico. Matematici, statistici e appassionati si interrogano su una domanda fondamentale: **le offerte del Dottore sono eque?** Oppure il Dottore calibra strategicamente le sue proposte per minimizzare i pagamenti da parte della produzione, sfruttando la pressione psicologica del momento?

L'obiettivo di questo progetto è rispondere a questa domanda con gli strumenti dell'**intelligenza artificiale**, in particolare con il **Machine Learning supervisionato**. A partire da un dataset di 89 puntate reali della stagione 2023/2024, si è addestrato un sistema di classificazione in grado di predire, dato lo stato della partita, se l'offerta del Dottore sarà *bassa*, *media* o *alta* rispetto al valore statisticamente atteso. Le predizioni permettono poi di stimare, in termini concreti, quanto il Dottore stia trattenendo rispetto a ciò che la matematica suggerirebbe.

Il progetto è stato sviluppato interamente in **Python** con la libreria *scikit-learn*, con una replica del modello principale in **KNIME Analytics Platform**.

2. Il Machine Learning

Il **Machine Learning** (ML) è una branca dell'Intelligenza Artificiale che studia algoritmi e tecniche capaci di *imparare dai dati*, migliorando le proprie prestazioni attraverso l'esperienza senza essere esplicitamente programmati per ogni caso specifico [1].

2.1. Tipologie di apprendimento

Esistono tre paradigmi principali:

- **Apprendimento supervisionato**: il modello viene addestrato su un insieme di esempi etichettati, ossia coppie input-output già note. L'obiettivo è generalizzare la relazione appresa per predire l'output di nuovi dati. Questo progetto rientra in questa categoria.
- **Apprendimento non supervisionato**: il modello lavora su dati privi di etichette, cercando strutture e pattern nascosti (es. clustering, riduzione della dimensionalità).

- **Apprendimento per rinforzo:** un agente impara a prendere decisioni sequenziali interagendo con un ambiente, ricevendo ricompense o penalità in base alle azioni compiute.

2.2. Classificazione e regressione

Nell'apprendimento supervisionato si distinguono due tipi di problemi:

- **Regressione:** la variabile target è continua (es. prevedere l'esatto importo dell'offerta in €).
- **Classificazione:** la variabile target è categorica, cioè appartiene a un insieme finito di classi (es. classificare l'offerta come *Bassa*, *Media* o *Alta*).

Per questo progetto si è scelto l'approccio della **classificazione a tre classi**: la discretizzazione del problema lo rende più robusto alla varianza del dataset (relativamente piccolo) e i risultati sono più facilmente interpretabili e comunicabili.

2.3. Le fasi del processo ML

Un progetto di Machine Learning si articola in fasi ben definite, che ricalcano quelle seguite in questo lavoro:

1. **Raccolta e comprensione dei dati:** analisi esplorativa, identificazione delle feature rilevanti.
2. **Pre-processing:** pulizia dei dati, gestione dei valori mancanti, normalizzazione, costruzione della variabile target.
3. **Suddivisione train/test:** separazione dei dati in un insieme di addestramento e uno di valutazione, per stimare le prestazioni del modello su dati mai visti.
4. **Addestramento:** il modello apprende i parametri ottimali minimizzando una funzione di errore sul training set.
5. **Validazione:** la cross-validation stima la capacità di generalizzazione del modello.
6. **Valutazione:** misurazione delle prestazioni sul test set tramite metriche standard (accuracy, precision, recall, F1-score).

3. Il Dataset

3.1. Fonte e raccolta dei dati

I dati utilizzati provengono dal repository open-source github.com/luzo7/affari-tuoi, realizzato da un appassionato del programma che ha sviluppato un software specifico per registrare manualmente, puntata per puntata, ogni azione di gioco. Il dataset copre un arco temporale di circa tre mesi e mezzo, con puntate dal **12 febbraio 2024** al **20 maggio 2024**.

I dati grezzi sono memorizzati in formato JSON e descrivono la sequenza completa di eventi di ogni partita: aperture di pacchi, offerte del Dottore, accettazioni/rifiuti e tipo di fine partita.

3.2. Struttura e statistiche descrittive

Tabella 1. Statistiche descrittive del dataset

Attributo	Valore
Numero di puntate	89
Totale offerte del Dottore	386
Offerte accettate	41 (10,6%)
Offerte rifiutate	342 (88,6%)
Media valore offerta	30.770 €
Media valore atteso	41.007 €
Ratio medio offerta/VA	0,789
Ratio minimo	0,351
Ratio massimo	1,488

Il **ratio offerta/valore atteso** è la metrica centrale del progetto. È definito come:

$$\text{ratio} = \frac{\text{offerta del Dottore}}{\mathbb{E}[\text{premi rimasti}]} = \frac{\text{offerta del Dottore}}{\frac{1}{n} \sum_{i=1}^n v_i} \quad (1)$$

dove n è il numero di pacchi ancora in gioco e v_i è il valore del pacco i . Un ratio pari a 1 significa che il Dottore offre esattamente il valore atteso matematico; valori inferiori indicano un’offerta penalizzante per il concorrente.

I dati mostrano che in media il Dottore offre l’**78,9%** del valore atteso, trattenendo quindi circa il 21% come “margine”. Questo gap non è uniforme nel corso della partita, come evidenziato nella Tabella 2.

Tabella 2. Comportamento del Dottore per turno di offerta

Turno	N. offerte	Ratio medio	Dev. std	Trattenuto medio (€)
1°	89	0,850	0,146	6.112
2°	86	0,751	0,153	11.113
3°	84	0,729	0,188	11.765
4°	67	0,776	0,199	12.218
5°	40	0,833	0,234	11.209
6°	16	0,894	0,324	9.879
7°	4	0,893	0,270	9.590

Emerge un pattern inequivocabile: il Dottore è più generoso nella prima offerta (ratio 0,850) per “agganciare” il concorrente, diventa più aggressivo al terzo e quarto turno (ratio 0,729–0,776, quando la pressione psicologica è massima e i pacchi ad alto valore sono ancora molti), per poi tornare a offrire di più nelle fasi finali quando il numero di pacchi rimasti è ridotto.

3.3. Analisi esplorativa

I tipi di conclusione delle 89 puntate sono distribuiti come segue: 40 puntate si concludono con l'**accettazione di un'offerta** (44,9%), 28 con l'**apertura del proprio pacco** (31,5%) e 21 grazie alla **Regione fortunata** (23,6%).

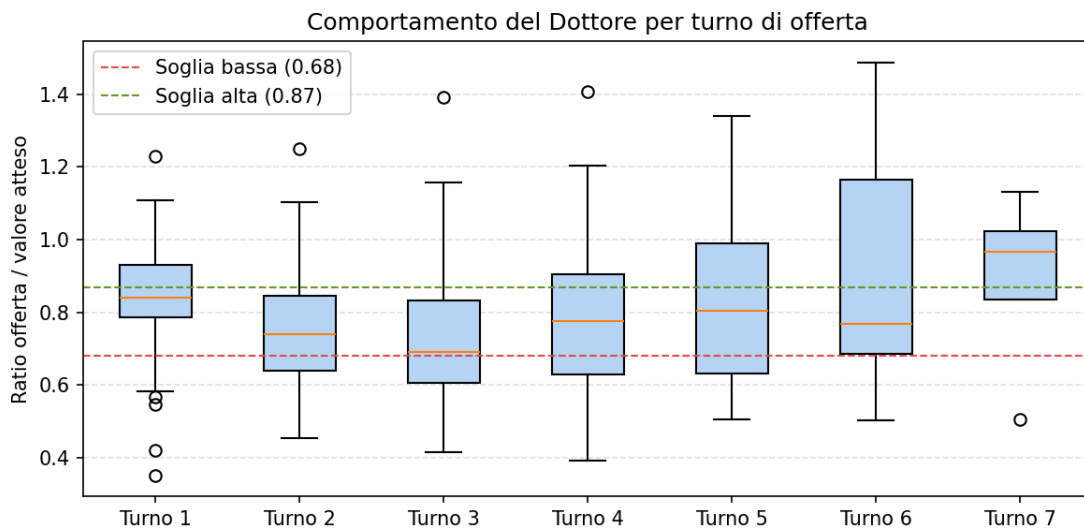


Figura 1. Distribuzione del ratio offerta/valore atteso per turno. Le linee tratteggiate indicano le soglie di classificazione: sotto 0,68 (rosso) l’offerta è classificata come *Bassa*, sopra 0,87 (verde) come *Alta*.

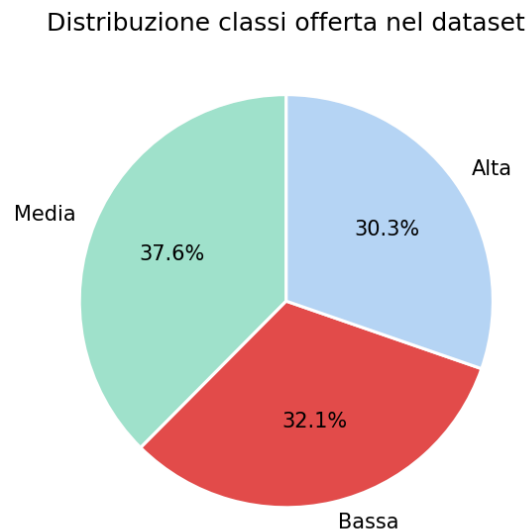


Figura 2. Distribuzione delle tre classi nel dataset (386 offerte totali).

4. Feature Engineering e Pre-processing

4.1. Estrazione delle feature

A partire dal file JSON grezzo, si è ricostruito lo **stato della partita in corrispondenza di ogni offerta**: per ogni azione di apertura si rimuove il valore corrispondente dal pool dei 21 premi disponibili, aggiornando dinamicamente le statistiche. Le feature calcolate per ciascuna delle 386 offerte sono riassunte nella Tabella 3.

Tabella 3. Feature di input utilizzate dai modelli

Feature	Tipo	Descrizione
num_offerta	Intero	Numero progressivo dell'offerta nella puntata (1-7)
n_pacchi_rimasti	Intero	Numero di pacchi ancora in gioco (3-15)
valore_atteso	Float	Media aritmetica dei premi rimasti (€)
mediana_rimasti	Float	Mediana dei premi rimasti, meno sensibile agli outlier
max_rimasto	Intero	Premio massimo ancora in gioco (€)
min_rimasto	Intero	Premio minimo ancora in gioco (€)
std_rimasti	Float	Deviazione standard dei premi rimasti
n_valori_alti	Intero	Numero di pacchi con valore ≥ 50.000 €

Non sono stati rilevati **valori mancanti** nel dataset. Ogni offerta è completamente descritta dalle otto feature sopra elencate.

4.2. Costruzione della variabile target

La variabile dipendente è il **tipo di offerta**, classificata in tre categorie sulla base del ratio definito nella (1). Le soglie sono state calibrate sulla distribuzione empirica dei dati, prendendo come riferimento i percentili P_{32} e P_{70} , in modo da ottenere classi il più possibile bilanciate:

- **Bassa** (classe 0): ratio $< 0,68 \Rightarrow 124$ offerte (32,1%)
- **Media** (classe 1): $0,68 \leq \text{ratio} \leq 0,87 \Rightarrow 145$ offerte (37,6%)
- **Alta** (classe 2): ratio $> 0,87 \Rightarrow 117$ offerte (30,3%)

4.3. Normalizzazione e suddivisione del dataset

Le feature numeriche sono state normalizzate tramite **MinMaxScaler**, che mappa ogni valore nell'intervallo $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2)$$

La normalizzazione è inclusa all'interno delle pipeline di scikit-learn, in modo da essere ricalcolata esclusivamente sui dati di addestramento in ogni fold della cross-validation, **evitando il data leakage**.

Il dataset è stato suddiviso con proporzione **75% train / 25% test**, con stratificazione sulla variabile target per preservare la distribuzione delle classi:

- Training set: **289 campioni**
- Test set: **97 campioni**

5. Modelli di Classificazione

Sono stati addestrati e confrontati tre classificatori supervisionati, scelti per la loro complementarità teorica e per la loro rilevanza nel contesto del corso.

5.1. Decision Tree

L'**albero decisionale** è un classificatore che costruisce un modello a struttura gerarchica: ogni nodo interno rappresenta un test su una feature, ogni ramo è l'esito del test, ogni foglia è una classe [2]. La predizione si ottiene percorrendo l'albero dalla radice alla foglia.

Il criterio di split usato è la **Gini impurity**, che misura la probabilità che un campione scelto casualmente venga classificato in modo errato:

$$G = 1 - \sum_{k=1}^K p_k^2 \quad (3)$$

dove p_k è la proporzione di campioni di classe k nel nodo.

Il parametro `max_depth=5` limita la profondità dell'albero per **prevenire l'overfitting**: senza tale vincolo, il Decision Tree tende a memorizzare il training set senza generalizzare.

5.2. Random Forest

La **foresta casuale** è un metodo di *ensemble learning* che combina le predizioni di un numero elevato di alberi decisionali addestrati in modo indipendente [3]. L'idea alla base è il principio della **saggezza della folla**: modelli diversi commettono errori diversi, e la loro aggregazione (per votazione a maggioranza nel caso della classificazione) riduce significativamente la varianza.

La diversità tra gli alberi è garantita da due meccanismi:

1. **Bagging** (*Bootstrap Aggregating*): ogni albero è addestrato su un campione con ripetizione (*bootstrap*) del training set.

2. **Feature randomness**: ad ogni split, solo un sottoinsieme casuale di $\sqrt{p}$ feature viene considerato (con p il numero totale di feature).

Nel progetto si è usata una foresta di `n_estimators=100` alberi. La Random Forest fornisce inoltre la **feature importance**, una misura di quanto ogni variabile contribuisce alla riduzione dell'impurità media attraverso tutti gli alberi.

5.3. Naive Bayes

Il classificatore **Naive Bayes** sfrutta il teorema di Bayes per stimare la probabilità a posteriori di ogni classe dato un vettore di feature:

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k) P(C_k)}{P(\mathbf{x})} \quad (4)$$

L'assunzione "naive" è la **indipendenza condizionale** tra le feature dato la classe: $P(\mathbf{x} | C_k) = \prod_j P(x_j | C_k)$. Nella variante **Gaussiana** adottata, si assume che ogni feature, condizionatamente alla classe, segua una distribuzione normale: $P(x_j | C_k) = \mathcal{N}(\mu_{jk}, \sigma_{jk}^2)$.

Nonostante la semplicità dell'assunzione (raramente soddisfatta in pratica), Naive Bayes è noto per la sua robustezza e velocità di addestramento, specialmente su dataset di dimensioni ridotte.

6. Implementazione

6.1. Implementazione Python

Il progetto è stato interamente implementato in **Python 3**, facendo uso delle seguenti librerie principali:

- `json`, `numpy`, `pandas`: lettura del dataset e manipolazione dei dati;
- `scikit-learn`: pre-processing, addestramento, cross-validation e metriche;
- `matplotlib`: generazione dei grafici.

L'architettura del codice segue il paradigma delle **pipeline** di scikit-learn, che incapsulano in un unico oggetto la sequenza di trasformazioni e il classificatore finale. Questo approccio garantisce l'assenza di data leakage durante la cross-validation.

```
1 from sklearn.pipeline import make_pipeline
2 from sklearn.preprocessing import MinMaxScaler
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.naive_bayes import GaussianNB
```

```
6 from sklearn.model_selection import KFold, cross_val_score
7
8 # Ogni pipeline: normalizzazione + classificatore
9 pipelines = {
10     "Decision Tree": make_pipeline(
11         MinMaxScaler(),
12         DecisionTreeClassifier(random_state=42, max_depth=5)
13     ),
14     "Random Forest": make_pipeline(
15         MinMaxScaler(),
16         RandomForestClassifier(n_estimators=100,
17                                random_state=42)
18     ),
19     "Naive Bayes": make_pipeline(
20         MinMaxScaler(),
21         GaussianNB()
22     )
23 }
24
25 # Cross-validation stratificata con 10 fold
26 kfold = KFold(n_splits=10, shuffle=True, random_state=42)
27
28 for nome, pipeline in pipelines.items():
29     scores = cross_val_score(
30         pipeline, X_train, y_train,
31         cv=kfold, scoring="accuracy"
32     )
33     print(f"{nome}: media={scores.mean():.3f}, std={scores.
34           std():.3f}")
```

Listing 1. Definizione delle pipeline e cross-validation

6.2. Implementazione KNIME

La Random Forest è stata replicata in **KNIME Analytics Platform** (versione 5), una piattaforma open-source per l'analisi visuale dei dati basata su flussi di lavoro a nodi. L'implementazione KNIME ha lo scopo principale di fornire una **rappresentazione grafica della pipeline**, particolarmente utile per comunicare il processo a un pubblico non tecnico, e di verificare la correttezza logica della sequenza di operazioni.

Il workflow, mostrato in Figura 3, comprende i seguenti 7 nodi collegati in sequenza:

1. **CSV Reader**: importazione del file `dataset_offerte.csv` generato dallo script Python (386 righe, 12 colonne);
2. **Column Filter**: selezione delle 8 feature di input e della colonna target `classe_offerta`;
3. **Normalizer** (Min-Max): normalizzazione nell'intervallo $[0, 1]$ delle 8 feature numeriche, escludendo il target;
4. **Row Sampler**: campionamento stratificato del 75% dei dati per il training (seed 42);
5. **Random Forest Learner**: addestramento del modello con 100 alberi, colonna target `classe_offerta`;
6. **Random Forest Predictor**: applicazione del modello addestrato ai dati in ingresso;
7. **Scorer**: calcolo della matrice di confusione e delle statistiche di classificazione.

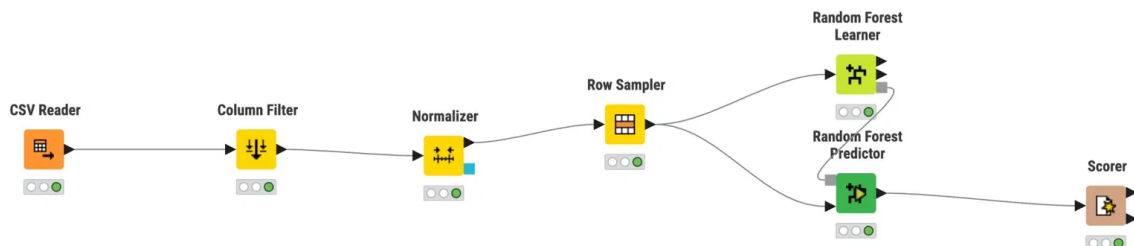


Figura 3. Workflow KNIME 5 per la classificazione delle offerte del Dottore. Da sinistra: CSV Reader → Column Filter → Normalizer → Row Sampler → Random Forest Learner / Predictor → Scorer.

Nota sul confronto con Python. A differenza dell'implementazione Python, in KNIME 5 il nodo *Row Sampler* dispone di una sola porta di uscita: produce il 75% dei dati destinato al training, ma non espone separatamente il restante 25% come test set indipendente. Di conseguenza, il Predictor valuta lo stesso insieme su cui il Learner è stato addestrato, producendo un'accuracy artificialmente elevata (99,3%) non comparabile con i risultati Python. L'implementazione KNIME è pertanto da intendersi come **strumento di visualizzazione della pipeline** e non come riferimento per la

valutazione delle prestazioni, ruolo assolto rigorosamente dall'implementazione Python con cross-validation a 10 fold e test set separato.

La Figura 4 mostra la matrice di confusione prodotta dallo Scorer KNIME, utile per verificare la correttezza logica del flusso.

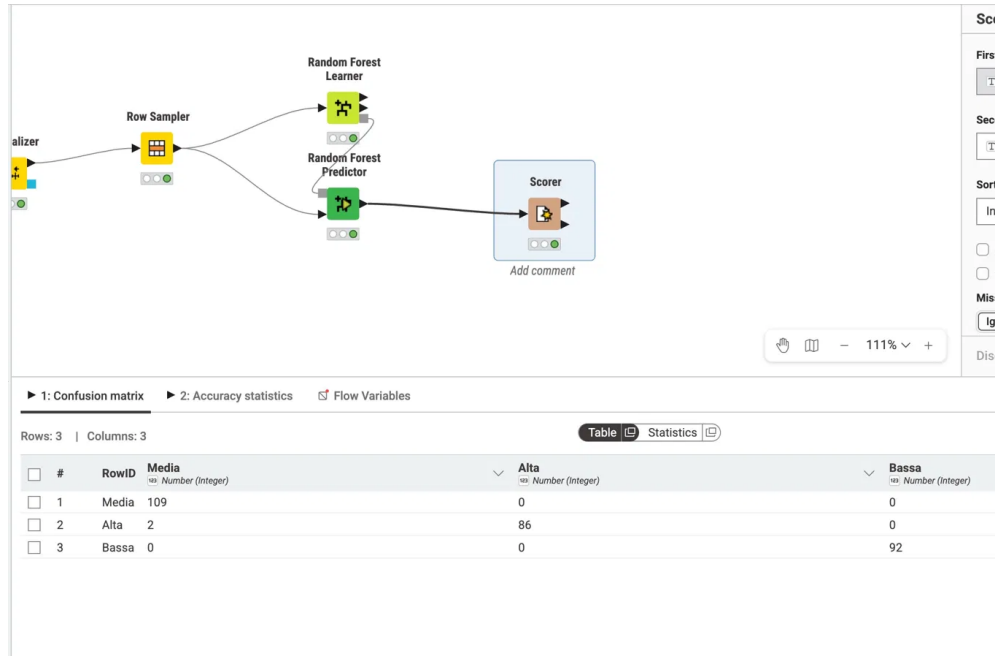


Figura 4. Matrice di confusione prodotta dallo Scorer KNIME. L'elevata accuracy (99,3%) è dovuta al fatto che il Predictor valuta dati già visti in fase di training — limitazione del nodo *Row Sampler* in KNIME 5. I risultati di riferimento sono quelli ottenuti in Python (Tabella 4).

7. Risultati e Valutazione

7.1. Metriche di valutazione

Per la valutazione dei modelli sono state usate le metriche standard della classificazione multiclasse.

L'**accuracy** misura la proporzione di predizioni corrette: $Acc = \frac{TP+TN}{TP+TN+FP+FN}$

Precision, **Recall** e **F1-score** vengono calcolate per ogni classe e poi mediate (*macro average*):

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (7)$$

7.2. Confronto tra modelli

Tabella 4. Risultati comparativi dei tre modelli su training (CV) e test set

Modello	Acc. test	CV media	CV std	F1 macro (test)
Decision Tree	0,598	0,564	0,104	0,597
Random Forest	0,577	0,622	0,084	0,581
Naive Bayes	0,556	0,561	0,059	0,554

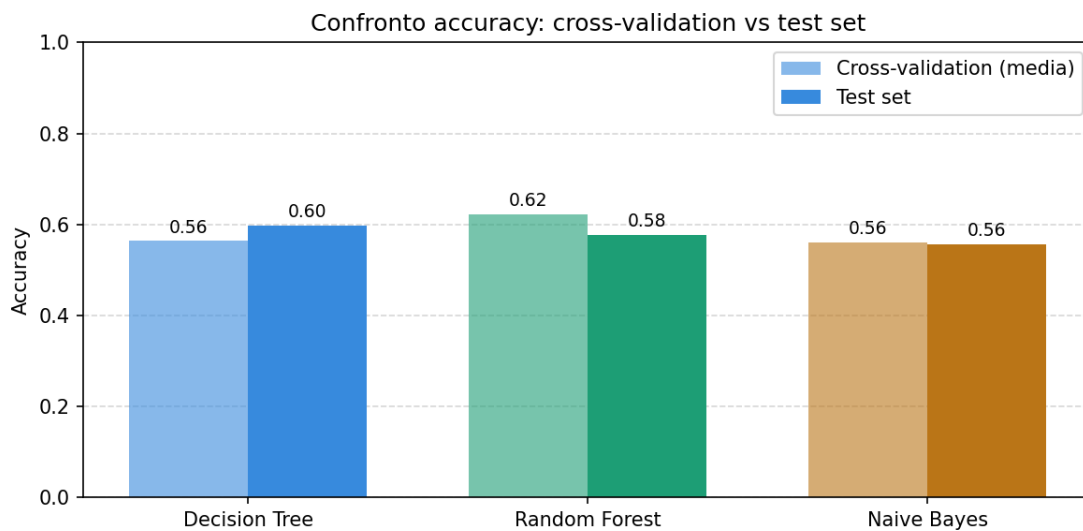


Figura 5. Confronto tra accuracy media in cross-validation e accuracy sul test set per i tre modelli.

I risultati sono **significativi**: in un problema di classificazione a tre classi equiprobabili, un classificatore casuale raggiungerebbe un'accuracy del 33%. I modelli addestrati raggiungono valori compresi tra il 55% e il 60%, raddoppiando di fatto la performance di una strategia casuale.

È importante notare la divergenza tra CV e test set per il Decision Tree (0,564 vs 0,598): questo comportamento è tipico di un modello con varianza elevata, che performa diversamente a seconda del sottoinsieme di dati su cui viene valutato. La Random Forest mostra invece la **CV più alta e la deviazione standard più bassa** (0,084), indicando maggiore stabilità e migliore capacità di generalizzazione, proprietà attese dall'ensemble learning.

7.3. Analisi per classe

Tabella 5. Precision, Recall e F1-score per classe — Random Forest

Classe	Precision	Recall	F1-score
Alta	0,633	0,655	0,644
Bassa	0,583	0,677	0,627
Media	0,516	0,432	0,471

La classe **Media** è la più difficile da classificare per tutti i modelli, con F1-score sistematicamente inferiore alle altre due classi. Questo risultato è interpretabile: le offerte “medie” sono quelle in cui il Dottore si comporta in modo meno prevedibile, variando il suo ratio in un intervallo ampio a seconda di fattori contestuali (regione del concorrente, dinamiche narrative della puntata, valore dei singoli pacchi rimasti) che non sono pienamente catturati dalle feature numeriche disponibili. Le classi **Alta** e **Bassa**, al contrario, corrispondono a comportamenti più stereotipati e riconoscibili.

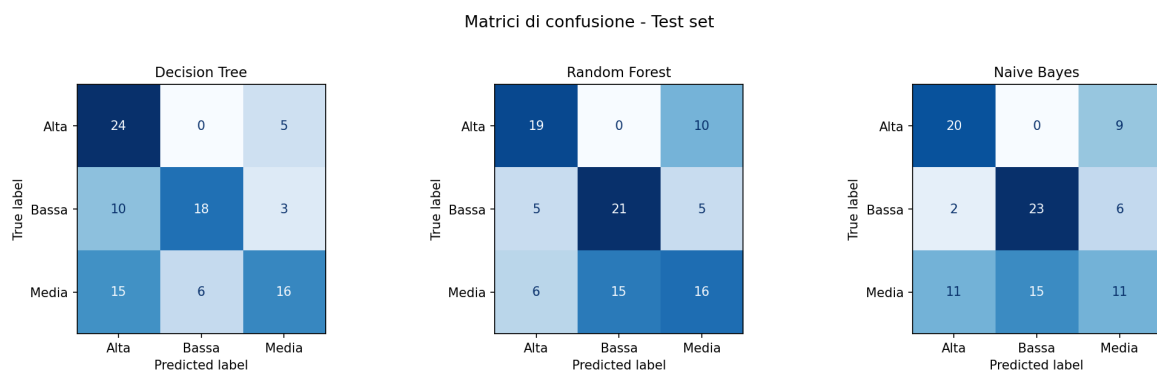


Figura 6. Matrici di confusione per i tre modelli sul test set. I valori sulla diagonale principale rappresentano le predizioni corrette.

7.4. Feature importance

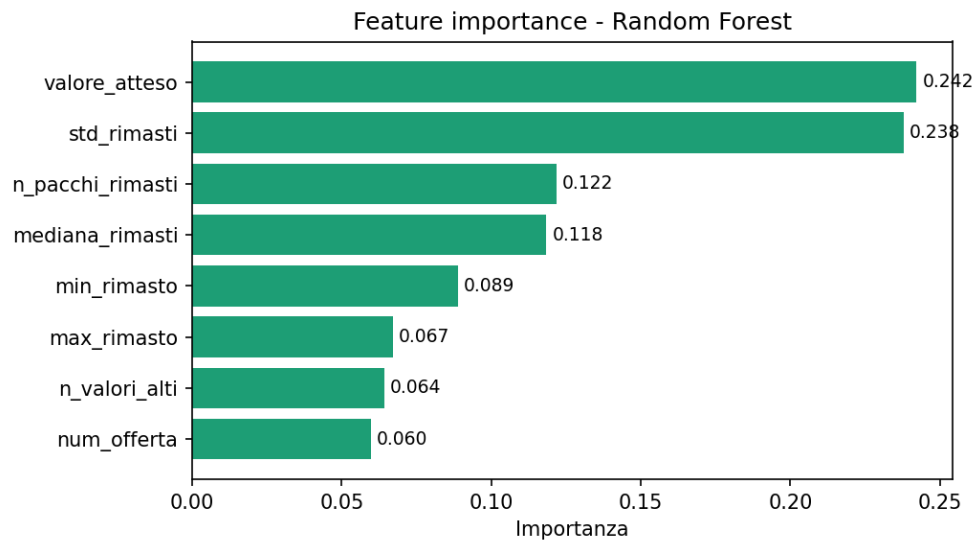


Figura 7. Importanza delle feature secondo la Random Forest, misurata come riduzione media dell'impurità di Gini.

Il grafico di feature importance rivela una gerarchia informativa chiara. Il **valore atteso** è di gran lunga la feature più predittiva (importanza $\approx 0,28$): è logico, perché è il denominatore del ratio e quindi la quantità che il Dottore deve conoscere per formulare qualsiasi offerta. A seguire troviamo **max_rimasto** e **std_rimasti**, che catturano il “rischio” per la produzione: quando è rimasto un pacco da 300.000 €, il Dottore ha tutto l'interesse a formulare un'offerta che spinga il concorrente a vendere. Il **numero di pacchi rimasti** e il **turno di offerta** completano il quadro, confermando il pattern temporale emerso nell'analisi esplorativa.

8. Interpretazione: quanto trattiene il Dottore?

Il vero contributo di questo progetto non è tanto tecnico quanto **narrativo**: le predizioni del modello permettono di rispondere alla domanda con cui si è aperto questo elaborato.

8.1. Quanto è distante l'offerta dalla matematica?

Nell'intera stagione analizzata, la media dell'offerta è stata di **30.770 €** a fronte di un valore atteso medio di **41.007 €**. Il Dottore ha quindi trattenuto in media **10.237 € per offerta** — circa il 25% del valore matematicamente atteso.

Il comportamento non è però uniforme. Gli esempi reali estratti dal test set illustrano i due estremi:

Offerta classificata come BASSA — esempio reale

Data: 20 aprile 2024 **Turno:** 3^a offerta

Pacchi rimasti: 7 **Valore atteso:** 57.305 €

Offerta del Dottore: 38.000 €

Trattenuto: 19.305 € (34% del valore atteso)

Il modello ha correttamente classificato questa offerta come *Bassa*, riconoscendo che il Dottore stava offrendo meno di due terzi del valore matematicamente atteso in un momento di alta pressione (terza offerta, molti pacchi ad alto valore ancora in gioco).

Offerta classificata come ALTA — esempio reale

Data: 5 marzo 2024 **Turno:** 6^a offerta

Pacchi rimasti: 3 **Valore atteso:** 40.333 €

Offerta del Dottore: 60.000 €

Ratio: 1,49 — il Dottore ha offerto il 49% in più del valore atteso

In questo caso il modello ha classificato correttamente l'offerta come *Alta*. Con soli 3 pacchi rimasti e una struttura di valori molto favorevole, la produzione ha preferito “comprare” la certezza evitando il rischio di pagare il massimo.

8.2. Implicazioni pratiche

Il modello non ha valore predittivo in senso assoluto (un concorrente non può attendere la risposta dell'algoritmo prima di decidere), ma ha un valore **analitico** e **conoscitivo** fondamentale: dimostra che le offerte del Dottore seguono una logica *apprendibile*, cioè che esistono pattern statisticamente riconoscibili nel suo comportamento.

In particolare:

- **Il Dottore non è casuale.** Se le offerte fossero formulate senza logica, nessun modello ML potrebbe superare significativamente il 33% di accuracy su un problema a tre classi. Il fatto che i nostri modelli raggiungano il 56–60% è la prova che un pattern esiste.
- **Il Dottore è più aggressivo a metà partita.** Il ratio scende al minimo al terzo turno (0,729) e aumenta nelle fasi finali (0,893), coerentemente con una strategia volta a sfruttare il momento di massima incertezza psicologica per il concorrente.
- **I premi alti influenzano l'offerta più dei premi bassi.** La feature importance indica che `max_rimasto` e `std_rimasti` sono tra le variabili più predittive, confermando che il Dottore calibra l'offerta soprattutto in funzione del “rischio” rappresentato dai pacchi ad alto valore ancora in gioco.

9. Conclusioni

Questo progetto ha dimostrato come le tecniche di **Machine Learning supervisionato** possano essere applicate con successo a un dominio inatteso come quello di un gioco televisivo, producendo risultati al tempo stesso tecnicamente validi e di interesse divulgativo.

I tre classificatori addestrati (Decision Tree, Random Forest, Naive Bayes) hanno raggiunto accuracy tra il 55% e il 60% su un problema a tre classi con baseline casuale al 33%, con la Random Forest che si è distinta per la maggiore stabilità in cross-validation ($\mu = 0,622$, $\sigma = 0,084$).

L'analisi delle feature importance ha evidenziato che il **valore atteso**, il **massimo rimasto** e la **deviazione standard** dei premi in gioco sono i fattori che più determinano il comportamento del Dottore, confermando l'ipotesi che le offerte seguano una logica di **minimizzazione del rischio** per la produzione.

Limiti del lavoro. Il principale limite è la **dimensione del dataset**: 386 offerte sono sufficienti per un progetto didattico, ma insufficienti per addestrare modelli ad alta capacità come le reti neurali. L'ampliamento del dataset a più stagioni di gioco potrebbe migliorare significativamente le performance. Ulteriori feature potenzialmente rilevanti — come la **regione di provenienza** del concorrente, il **mese** della puntata o una stima del **coinvolgimento narrativo** dell'episodio — non erano disponibili nel dataset utilizzato o avrebbero richiesto un processo di raccolta dati considerevolmente più complesso.

Sviluppi futuri. Alcune direzioni di ricerca meritano approfondimento:

- Estendere il dataset alle stagioni precedenti per aumentare la potenza statistica;
- Introdurre un modello di **regressione** per predire il valore esatto del ratio, anziché la sola classe;
- Esplorare tecniche di **spiegabilità** (SHAP values, LIME) per motivare le singole predizioni in modo interpretabile;
- Analizzare eventuali **differenze regionali** nel comportamento del Dottore, aspetto particolarmente discusso nel dibattito pubblico.

Considerazione finale. I risultati di questo progetto sono coerenti con le analisi statistiche pubblicate da altri ricercatori sullo stesso programma: le offerte del Dottore non sono casuali e seguono schemi riconoscibili. Il Machine Learning aggiunge a questa osservazione una dimensione predittiva: non solo il Dottore *si comporta* in modo

sistematico, ma quel comportamento è *apprendibile* da un algoritmo addestrato su dati storici.

Riferimenti bibliografici

- [1] T.M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
- [2] L. Breiman, J. Friedman, C.J. Stone, R.A. Olshen, *Classification and Regression Trees*, Chapman & Hall/CRC, 1984.
- [3] L. Breiman, “Random Forests”, *Machine Learning*, vol. 45, n. 1, pp. 5–32, 2001.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python”, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] luzo7, *Affari Tuoi Dataset*, GitHub, 2024. <https://github.com/luzo7/affari-tuoi>
- [6] P. Parenti, *Affari Tuoi Fairness Analysis*, GitHub, 2024. <https://github.com/PietroParenti/affari-tuoi-fairness>